Task 4 — Async React: Effects & API

Integrate the provided starter project https://github.com/leravanna/fake-api with a simulated asynchronous API using Promises, random delays, errors, and `localStorage`. You can implement similar mechanics, consider that your app should be deployed and fully operational.

Requirements:

- Extend the item details data to include at least 7 meaningful fields displayed in the UI.
- Load the item list using `useEffect`.
- Add Create, Edit, Remove functionality.
- Allow users to select an item and asynchronously load its details (extend details so more is loaded).
- Add an action that updates the selected item through the API (set anything like complete, make available, change status, etc).
- Handle loading, error, empty, and success states.
- Add a Reload button to refresh the list.
- Use correct `useEffect` dependencies and cleanup.
- Prevent race conditions: quickly switching between items must never display outdated details.
- Handle stale closures: capture the selected item's ID when starting completion, prevent conflicting actions, and ensure the response updates the correct item.
- Create a clean, responsive, and usable UI.

Restrictions: No other hooks than useState and useEffect, no custom hooks, no debouncing, no external fetching libraries, direct `localStorage` access from components

Submission: GitHub repository link and deployed application link.

Defense: Explain `useEffect`, dependencies, cleanup, race conditions, stale closures, and demonstrate correct behavior with delayed requests.